# Übungsblatt 2

## Typentheorie: Terme, Typen, Regeln

Skript: Abschnitte 3.1–3.5

Auf diesem Blatt arbeiten Sie mit den Regeln, nach denen der Kern prüft: Sie ordnen Typen in Universen ein, leiten ein Typurteil her, lesen Aussagen als Typen und Beweise als Terme, rechnen nach, was definitorisch gleich ist, und untersuchen einen selbst gebauten induktiven Typ. In Teil B schreiben Sie dieselben Terme zum ersten Mal in Lean – alles im Termmodus.

## Teil A: Auf Papier

### Aufgabe A1 (Universen)

Bestimmen Sie für jeden der folgenden Typen $T$ das Universum, in dem er liegt, also das Sort $w$ mit $\vdash T : \mathrm{Sort}\ w$, und geben Sie es als Prop oder Type $u$ an. Benutzen Sie die Regeln (Sort), (Var) und (Π-Form) aus Abschnitt 3.5 und zwei weitere Formationsregeln aus der Vorlesung: (ℕ-Form) liefert $\vdash \mathbb{N} : \mathrm{Type}$, und nach (=-Form) ist $a = b$ für Terme $a, b$ desselben Typs eine Aussage, also vom Typ Prop.

(a) $\mathbb{N} \to \mathrm{Prop}$

(b) $(n : \mathbb{N}) \to n + 0 = n$

(c) $(p : \mathrm{Prop}) \to p \to p$

(d) $(\alpha : \mathrm{Type}) \to \alpha \to \alpha$

(e) $\mathrm{Type} \to \mathrm{Prop}$

(f) Nach (App) darf man in (c) für $p$ die Aussage aus (c) selbst einsetzen, in (d) für $\alpha$ den Typ aus (d) dagegen nicht. Begründen Sie beides mit Ihren Ergebnissen. Welche Stelle der Regel (Π-Form) macht den Unterschied, und warum darf man bei Type nicht ebenso verfahren wie bei Prop?

*Hinweis:* Sort $0 = \mathrm{Prop}$ und Sort $(u + 1) = \mathrm{Type}\ u$; ferner $\mathrm{imax}\ u\ 0 = 0$ und sonst $\mathrm{imax}\ u\ v = \max(u, v)$. Kontrollieren können Sie Ihre Ergebnisse in Lean mit `#check`.

### Aufgabe A2 (Eine Typableitung)

Sei $\Gamma_0 := A : \mathrm{Prop},\ B : \mathrm{Prop},\ C : \mathrm{Prop}$.

(a) Leiten Sie mit den Regeln (Var), (λ) und (App) das Urteil

$$\Gamma_0 \vdash \lambda f.\, \lambda g.\, \lambda x.\, g\,(f\,x) : (A \to B) \to (B \to C) \to A \to C$$

her. Suchen Sie die Herleitung wie in der Vorlesung rückwärts und schreiben Sie sie als Baum auf. Abkürzungen wie $\Gamma_1 := \Gamma_0,\ f : A \to B$ sind erlaubt.

(b) Streichen Sie in Ihrem Baum die Terme. Welche Herleitung im natürlichen Schließen (Abschnitt 2.4) entsteht? Vergleichen Sie mit Ihrem Baum aus Blatt 1, Aufgabe A2 (c). Welches $\lambda$ entlässt welche Annahme? Welche Einträge des Kontexts an den Blättern werden dabei zu Annahmen im natürlichen Schließen, welche nicht?

(c) Geben Sie drei verschiedene Terme vom Typ $(\alpha \to \alpha) \to \alpha \to \alpha$ an, für einen Typ $\alpha$. Was liefern sie für $\alpha := \mathbb{N}$, angewandt auf die Nachfolgerfunktion und 0? Unter welchem Namen kennen Sie sie aus Abschnitt 2.4? Und was wird aus den drei Termen, wenn man für $\alpha$ eine Aussage $A : \mathrm{Prop}$ einsetzt?

*Hinweis:* Pfeile klammern nach rechts: Der Typ in (a) ist $(A \to B) \to \big((B \to C) \to (A \to C)\big)$.

**Aufgabe A3 (Aussagen als Typen)**

Seien $A, B, C$ Aussagen. Geben Sie zu jeder der folgenden Aussagen einen Term dieses Typs an. Bausteine sind $\lambda$, Anwendung und die Terme aus den Regeln für Paare und Summen in der Vorlesung, hier für $\wedge$ und $\vee$: für $a : A$ und $b : B$ das Paar $\langle a, b\rangle : A \wedge B$, für $h : A \wedge B$ die Projektionen $h.1 : A$ und $h.2 : B$; die Injektionen $\mathrm{inl}\ a : A \vee B$ und $\mathrm{inr}\ b : A \vee B$; für $h : A \vee B$ und $u : A \to D$, $v : B \to D$ mit einer Aussage $D$ die Fallunterscheidung $h.\mathrm{elim}\ u\ v : D$. Dabei ist $\neg A := A \to \mathrm{False}$, und ein Term für $P \leftrightarrow Q$ ist ein Paar aus einem Term für $P \to Q$ und einem für $Q \to P$.

(a) $(A \to B) \to A \wedge C \to B \wedge C$. Zeichnen Sie den Herleitungsbaum wie in der Vorlesung bei $\lambda h.\, h.2\ h.1$, an jedem Knoten sein Term.

(b) $\neg(A \vee B) \leftrightarrow \neg A \wedge \neg B$. Schreiben Sie zu jedem Baustein dazu, welcher Regel des natürlichen Schließens er entspricht.

(c) $\neg\neg\neg A \to \neg A$. Für $\neg\neg A \to A$ gibt es keinen solchen Term – hier schon.

(d) Für $\neg(A \wedge B) \to \neg A \vee \neg B$ finden Sie keinen. Erklären Sie informell, woran die Suche scheitert. Geben Sie dann einen Term an, der zusätzlich $\mathrm{em} : (P : \mathrm{Prop}) \to P \vee \neg P$ benutzen darf.

**Aufgabe A4 (Rechnen im Kern)**

Sei $\mathrm{zweimal} := \lambda f.\, \lambda x.\, f\,(f\,x)$ und $\mathrm{doppelt} := \lambda n.\, n + n$, beide für natürliche Zahlen. Schreiben Sie an jeden Reduktionsschritt die Regel ($\beta$, $\delta$, $\zeta$, $\iota$). Summen und Produkte fertiger Zahlen wie $3 + 3$ dürfen Sie am Ende in einem Schritt ausrechnen.

(a) Zeigen Sie $\mathrm{zweimal}\ \mathrm{doppelt}\ 3 \rightsquigarrow 12$ und $(\textbf{let}\ d := \mathrm{doppelt};\ d\,(d\,1)) \rightsquigarrow 4$.

(b) Sei $r := \mathrm{rec}\ 1\ s$ der Rekursor über $\mathbb{N}$ mit dem Schritt $s := \lambda_\ ih.\ 2 \cdot ih$, also $r\ 0 \rightsquigarrow 1$ und $r\,(n{+}1) \rightsquigarrow s\ n\ (r\ n)$. Zeigen Sie $r\ 3 \rightsquigarrow 8$. Welche Funktion berechnet $r$?

(c) In Lean sind $+$ und $\cdot$ auf $\mathbb{N}$ durch Rekursion über das *zweite* Argument definiert,

$$n + 0 \rightsquigarrow n, \qquad n + \mathrm{succ}\ m \rightsquigarrow \mathrm{succ}\,(n + m), \qquad n \cdot 0 \rightsquigarrow 0, \qquad n \cdot \mathrm{succ}\ m \rightsquigarrow n \cdot m + n,$$

und 1 ist succ 0. Welche der folgenden Gleichungen gelten für eine Variable $n : \mathbb{N}$ definitorisch, welche nur propositional? Begründen Sie durch Rechnen.

(i) $n + 1 = \mathrm{succ}\ n$ (ii) $1 + n = \mathrm{succ}\ n$ (iii) $0 \cdot n = 0$
(iv) $n \cdot 1 = n$ (v) $n \cdot 1 = 0 + n$

*Hinweis:* Eine durch Rekursion definierte Funktion rechnet nur weiter, wenn das Argument, über das sie rekursiert, mit einem Konstruktor beginnt. Da die Reduktion konfluent ist und stets endet, sind zwei Terme genau dann definitorisch gleich, wenn sie dieselbe Normalform haben (Abschnitt 3.5).

### Aufgabe A5 (Binärbäume)

Der induktive Typ Baum hat die Konstruktoren blatt : Baum und knoten : Baum → Baum → Baum. Durch Fallunterscheidung nach dem Konstruktor sind definiert:

$$\begin{aligned} \text{blaetter blatt} &= 1, & \text{blaetter}\,(\text{knoten } l\ r) &= \text{blaetter } l + \text{blaetter } r,\\ \text{innen blatt} &= 0, & \text{innen}\,(\text{knoten } l\ r) &= \text{innen } l + \text{innen } r + 1,\\ \text{spiegel blatt} &= \text{blatt}, & \text{spiegel}\,(\text{knoten } l\ r) &= \text{knoten}\,(\text{spiegel } r)\,(\text{spiegel } l). \end{aligned}$$

(a) Zeichnen Sie $t := \text{knoten}\,(\text{knoten blatt blatt})\,(\text{knoten blatt}\,(\text{knoten blatt blatt}))$ und berechnen Sie blaetter $t$, innen $t$ und spiegel $t$.

(b) Geben Sie den Typ des Rekursors Baum.rec an und seine beiden $\iota$-Regeln. Was ergibt sich, wenn das Motiv Werte in Prop hat? Schreiben Sie blaetter als Anwendung des Rekursors.

(c) Zeigen Sie durch strukturelle Induktion: blaetter $t = \text{innen } t + 1$ für alle $t$ : Baum.

(d) Zeigen Sie ebenso spiegel (spiegel $t$) $= t$ für alle $t$. Welche Schritte Ihres Beweises sind Rechnen, also definitorisch, und wo geht die Induktionsvoraussetzung ein?

*Hinweis:* Für $\mathbb{N}$ hat der Rekursor den Typ Nat.rec : $\{\mathit{motive} : \mathbb{N} \to \text{Sort } u\} \to \mathit{motive}\ 0 \to \big((n : \mathbb{N}) \to \mathit{motive}\ n \to \mathit{motive}\,(n+1)\big) \to (t : \mathbb{N}) \to \mathit{motive}\ t$.

## Teil B: In Lean

**So arbeiten Sie mit der Datei.** Öffnen Sie `lean/Blatt02.lean` in VS Code mit der Lean-4-Erweiterung, und zwar innerhalb dieses Repositorys: Dort ist Mathlib eingebunden. Jedes `sorry` ist eine Lücke; ersetzen Sie es durch einen Term. Steht der Cursor in einer Lücke, zeigt der Infoview rechts den Kontext, also die Annahmen, und den Typ, der an dieser Stelle verlangt ist. Fertig sind Sie, wenn Lean keine Warnung `declaration uses 'sorry'` mehr meldet. Sonderzeichen tippt man mit einem Backslash und einem Kürzel, das die Erweiterung durch das Zeichen ersetzt: `\<` und `\>` ergeben ⟨ und ⟩, `\2` ergibt den Index in $\texttt{congrArg}_2$. Zeigt man mit der Maus auf ein Zeichen in der Datei, nennt die Erweiterung sein Kürzel.

Alle Aufgaben gehen im *Termmodus*: Sie schreiben den Term direkt hin, mit `fun`, Anwendung, `⟨_, _⟩` und den Namen aus den Hinweisen. Den Taktikmodus aus Vorlesung 2 brauchen Sie hier nicht; einzige Ausnahme ist `by decide` in B1. Wenn Sie nicht weiterwissen, schreiben Sie an die fragliche Stelle einen Unterstrich `_`: Lean meldet dann einen Fehler, der Kontext und erwarteten Typ dieser Stelle nennt. Bearbeiten Sie jeweils zuerst die Teil-A-Aufgabe, die formalisiert wird.

### Aufgabe B1 (Formalisierung von A2)

In der Datei steht das Urteil aus A2 (a) als Satz `kette`; dieselbe Typform für Daten ist die Definition `nacheinander`. Geben Sie beide Male den Term an. Zu A2 (c) ist `null` schon definiert, und zwar für jedes Universum `Sort u`. Ergänzen Sie `eins` und `zwei`, und zeigen Sie dann: `eins` ℕ und `zwei` ℕ zählen bis 1 und 2; `null` ℕ und `zwei` ℕ sind verschiedene Programme; für eine Aussage `A` sind `null A` und `zwei A` gleich. Die Ungleichung erfasst nur den Fall $\alpha := \mathbb{N}$: Hat $\alpha$ nur ein Element, so sind `null` und `zwei` dort dieselbe Funktion.

*Hinweis:* $a \neq b$ bedeutet $\neg(a = b)$, ein Beweis ist also eine Funktion, die aus $h$ : null ℕ = zwei ℕ einen Widerspruch macht. `congrFun h a` macht aus $h : f = g$ die Gleichung $f\,a = g\,a$; zweimal angewandt entsteht eine Gleichung zwischen zwei Zahlen. `absurd` schließt aus $e : P$ und $n : \neg P$ auf alles, und eine falsche Gleichung zwischen Zahlen widerlegt `by decide`.

Lean lean/Blatt02.lean

```
/-- Das Urteil aus A2 (a). Lean prueft genau
‘A : Prop, B : Prop, C : Prop ⊢ t : (A → B) → (B → C) → A → C‘. -/
theorem kette (A B C : Prop) : (A → B) → (B → C) → A → C :=
  sorry

/-- Derselbe Typ fuer Daten: erst ‘f‘ anwenden, dann ‘g‘. -/
def nacheinander (α β γ : Type) : (α → β) → (β → γ) → α → γ :=
  sorry

/- Die Terme aus A2 (c), fuer jedes Universum ‘Sort u‘. -/
universe u

def null (α : Sort u) : (α → α) → α → α := fun _ x => x

def eins (α : Sort u) : (α → α) → α → α :=
  sorry

def zwei (α : Sort u) : (α → α) → α → α :=
  sorry

/- Als Programme, mit α := ℕ: ‘eins‘ und ‘zwei‘ zaehlen bis 1 bzw. 2 ... -/
example : eins ℕ Nat.succ 0 = 1 :=
  sorry

example : zwei ℕ Nat.succ 0 = 2 :=
  sorry

/- ... und ‘null ℕ‘ ist ein anderes Programm als ‘zwei ℕ‘.
   ‘congrFun h a‘ macht aus ‘h : f = g‘ die Gleichung ‘f a = g a‘. -/
example : null ℕ ≠ zwei ℕ :=
  sorry

/- Als Beweise, mit einer Aussage A fuer α: -/
example (A : Prop) : null A = zwei A :=
  sorry
```

### Aufgabe B2 (Formalisierung von A3)

Die Aussagen aus A3 stehen in der Datei; gesucht sind Ihre Terme, in Lean geschrieben. Die Bausteine heißen dort `⟨_, _⟩` für das Paar, `h.1` und `h.2` für die Projektionen, `Or.inl` und `Or.inr` für die Injektionen und `h.elim` für die Fallunterscheidung; auch ein Beweis von ↔ ist ein Paar `⟨_, _⟩`. Für (d) steht `Classical.em A` zur Verfügung, ein Beweis von `A ∨ ¬A`. Lassen Sie sich danach mit `#print axioms` anzeigen, worauf `dreifach` und `klassisch` beruhen, und vergleichen Sie mit Abschnitt 3.4.

Lean lean/Blatt02.lean

```
/-- (a) -/
theorem und_symm (A B : Prop) : A ∧ B → B ∧ A :=
  sorry

/-- (b) De Morgan, in beiden Richtungen konstruktiv -/
theorem de_morgan (A B : Prop) : ¬(A ∨ B) ↔ ¬A ∧ ¬B :=
  sorry

/-- (c) Dreifache Negation -/
theorem dreifach (A : Prop) : ¬¬¬A → ¬A :=
  sorry
```

```
/-- (d) Die andere De-Morgan-Regel, mit ‘Classical.em A : A ∨ ¬A‘ -/
theorem klassisch (A B : Prop) : ¬(A ∧ B) → ¬A ∨ ¬B :=
  sorry

/- Worauf beruhen die beiden letzten Saetze? Ueber die Typentheorie hinaus
   braucht Mathlib nur ‘propext‘, ‘Classical.choice‘ und ‘Quot.sound‘. -/
#print axioms dreifach
#print axioms klassisch
```

### Aufgabe B3 (Formalisierung von A4)

`zweimal` und `doppelt` sind wie in A4 definiert. Bestätigen Sie die Rechnungen aus A4 (a) und (b), und beweisen Sie die Gleichungen aus A4 (c): mit `rfl`, wo sie definitorisch gelten – dann rechnet der Kern selbst –, und sonst mit einem der Sätze `Nat.add_zero`, `Nat.zero_add`, `Nat.one_add`, `Nat.mul_zero`, `Nat.zero_mul` und `Nat.one_mul`, angewandt auf `n`. Was ein Satz besagt, zeigt `#check`. Nicht jeder passt; einer passt, obwohl er auf den ersten Blick nichts mit seiner Gleichung zu tun hat. Warum akzeptiert Lean ihn?

**Lean** `lean/Blatt02.lean`

```
def zweimal : (ℕ → ℕ) → ℕ → ℕ := fun f x => f (f x)
def doppelt : ℕ → ℕ := fun n => n + n

/- Die Rechnungen aus A4 (a) und (b) -/
example : zweimal doppelt 3 = 12 :=
  sorry

example : (let d := doppelt; d (d 1)) = 4 :=
  sorry

example : Nat.rec (motive := fun _ => ℕ) 1 (fun _ ih => 2 * ih) 3 = 8 :=
  sorry

/- Die Gleichungen aus A4 (c): ‘rfl‘, wo sie definitorisch gelten, sonst
   einer der Saetze ‘Nat.add_zero‘, ‘Nat.zero_add‘, ‘Nat.one_add‘,
   ‘Nat.mul_zero‘, ‘Nat.zero_mul‘, ‘Nat.one_mul‘ (nicht jeder passt). -/
example (n : ℕ) : n + 1 = Nat.succ n :=
  sorry

example (n : ℕ) : 1 + n = Nat.succ n :=
  sorry

example (n : ℕ) : 0 * n = 0 :=
  sorry

example (n : ℕ) : n * 1 = n :=
  sorry

example (n : ℕ) : n * 1 = 0 + n :=
  sorry
```

### Aufgabe B4 (Formalisierung von A5)

Die Datei enthält den Typ `Baum` und die Funktion `blaetter` aus A5.

(a) Definieren Sie `innen` und `spiegel` wie in A5, mit einem Fall pro Konstruktor nach dem Muster von `blaetter`.

(b) Bestätigen Sie mit `rfl` die Werte innen $t$ und spiegel $t$ aus A5 (a) und die Aussagen von A5 (c) und (d) für den Baum `beispiel`; das ist $t$ aus A5 (a). Die ersten beiden Gleichungen prüfen zugleich Ihre Definitionen. Mit `#check @Baum.rec` können Sie Ihre Antwort zu A5 (b) vergleichen.

(c) Beweisen Sie `spiegel_spiegel`, also A5 (d) für alle Bäume.

**Lean** `lean/Blatt02.lean`

```
inductive Baum where
  | blatt : Baum
  | knoten : Baum → Baum → Baum

namespace Baum

/-- Anzahl der Blaetter: ein Fall pro Konstruktor, wie bei ‘MeinNat.add‘. -/
def blaetter : Baum → ℕ :=
  fun
  | .blatt => 1
  | .knoten l r => blaetter l + blaetter r

/-- (a) Anzahl der inneren Knoten -/
def innen : Baum → ℕ :=
  sorry

/-- (a) Spiegeln: linken und rechten Teilbaum tauschen, auf allen Ebenen -/
def spiegel : Baum → Baum :=
  sorry

/-- Der Baum ‘t‘ aus A5 (a) -/
def beispiel : Baum :=
  .knoten (.knoten .blatt .blatt) (.knoten .blatt (.knoten .blatt .blatt))

/- (b) Zuerst die Werte aus A5 (a) -- das prueft auch Ihre Definitionen. -/
example : innen beispiel = 4 :=
  sorry

example : spiegel beispiel =
    .knoten (.knoten (.knoten .blatt .blatt) .blatt) (.knoten .blatt .blatt) :=
  sorry

/- Dann A5 (c) und (d) am Beispiel -- der Kern rechnet beide Seiten aus. -/
example : blaetter beispiel = innen beispiel + 1 :=
  sorry

example : spiegel (spiegel beispiel) = beispiel :=
  sorry

#check @Baum.rec    -- zum Vergleich mit A5 (b)
```

Zu (c): Ein Induktionsbeweis ist in Lean eine rekursive Funktion wie `blaetter`; die Aufrufe `spiegel_spiegel l` und `spiegel_spiegel r` sind die Induktionsvoraussetzungen. `congrArg`$_2$ `knoten` macht aus $a = b$ und $c = d$ die Gleichung knoten $a\ c =$ knoten $b\ d$. A5 (c) für alle Bäume lassen wir weg: Dort braucht der Schritt Rechengesetze für $+$, bequem erst mit Taktiken wie `omega`.

**Lean** `lean/Blatt02.lean`

```
/-- (c) A5 (d) fuer alle Baeume. Im Fall ‘.knoten l r‘ sind
‘spiegel_spiegel l‘ und ‘spiegel_spiegel r‘ die Induktionsvoraussetzungen. -/
theorem spiegel_spiegel : (t : Baum) → spiegel (spiegel t) = t
  | .blatt =>
    sorry
  | .knoten l r =>
    sorry
end Baum
```